

LAB 8: Test-Driven Development with AI – Generating and Working with Test Cases

ASSIGNMENT-8.2

NAME : K.SAI VARDHAN REDDY

ROLL NO. : 2503A51266

BATCH - 43

1. Lab Objectives

- Introduce Test-Driven Development (TDD) using AI-assisted test generation.
- Generate test cases before writing the actual implementation.
- Validate code using Python's unittest framework.
- Handle invalid inputs and boundary cases safely.
- Analyze AI-generated tests and compare them with expected behavior.

2. TDD Approach Used

The Red–Green–Refactor approach is followed. First, expected behavior is converted into test cases (Red). Next, the minimum implementation needed to satisfy the tests is written (Green). Finally, the implementation is reviewed and improved for readability, validation, and error handling (Refactor).

3. Task 1 – Even/Odd Number Validator

Function: `is_even(n)`

Requirements: Input must be an integer; zero, negative numbers, and large integers must be handled.

AI Prompt Used

Generate unittest test cases for a Python function `is_even(n)`.

Requirements:

1. Input must be an integer.
2. Test even and odd positive numbers.

3. Test zero and negative numbers.
 4. Test a very large integer.
 5. Test invalid non-integer input and expect `TypeError`.
- Return only a complete unittest test class with clear test names.

AI-Generated Test Cases

```
import unittest

class TestIsEven(unittest.TestCase):
    def test_even_number(self):
        self.assertTrue(is_even(2))

    def test_odd_number(self):
        self.assertFalse(is_even(7))

    def test_zero(self):
        self.assertTrue(is_even(0))

    def test_negative_even(self):
        self.assertTrue(is_even(-4))

    def test_negative_odd(self):
        self.assertFalse(is_even(-5))

    def test_large_integer(self):
        self.assertTrue(is_even(10**100))

    def test_invalid_input(self):
        with self.assertRaises(TypeError):
            is_even("2")
```

Implementation

```
def is_even(n):
    if not isinstance(n, int) or isinstance(n, bool):
        raise TypeError("Input must be an integer")
    return n % 2 == 0
```

Code Explanation

- `isinstance(n, int)` verifies that the input is an integer.
- Boolean values are excluded because `bool` is a subclass of `int` in Python.
- The modulo operator (%) checks whether the remainder after division by 2 is zero.
- The same logic correctly handles zero, negative integers, and arbitrarily large Python integers.

Expected Output

```
-----
Ran 7 tests in <time>

OK
Result: All Task 1 tests pass.
```

4. Task 2 – String Case Converter

Functions: `to_uppercase(text)` and `to_lowercase(text)`

Requirements: Empty strings, mixed-case strings, and invalid inputs such as numbers or `None` must be safely handled. This solution raises `TypeError` for invalid inputs.

AI Prompt Used

Generate unittest test cases for Python functions `to_uppercase(text)` and `to_lowercase(text)`.

Requirements:

1. Test normal strings.
 2. Test mixed-case strings.
 3. Test empty strings.
 4. Test strings containing spaces and numbers.
 5. Test `None` and numeric inputs; expect `TypeError`.
- Return a complete unittest test class.

AI-Generated Test Cases

```
import unittest

class TestCaseConverter(unittest.TestCase):
    def test_uppercase_normal(self):
        self.assertEqual(to_uppercase("ai coding"), "AI CODING")

    def test_lowercase_normal(self):
        self.assertEqual(to_lowercase("TEST"), "test")

    def test_uppercase_empty(self):
        self.assertEqual(to_uppercase(""), "")

    def test_lowercase_empty(self):
        self.assertEqual(to_lowercase(""), "")

    def test_uppercase_mixed_case(self):
        self.assertEqual(to_uppercase("PyThOn 123"), "PYTHON 123")

    def test_lowercase_mixed_case(self):
        self.assertEqual(to_lowercase("PyThOn 123"), "python 123")

    def test_uppercase_none(self):
        with self.assertRaises(TypeError):
            to_uppercase(None)

    def test_lowercase_number(self):
        with self.assertRaises(TypeError):
            to_lowercase(123)
```

Implementation

```
def to_uppercase(text):
    if not isinstance(text, str):
        raise TypeError("Input must be a string")
    return text.upper()
```

```
def to_lowercase(text):
    if not isinstance(text, str):
        raise TypeError("Input must be a string")
    return text.lower()
```

Code Explanation

- Both functions first verify that the input is a string.
- Invalid inputs such as None and numbers raise a clear TypeError.
- upper() converts alphabetic characters to uppercase while preserving spaces and numbers.
- lower() converts alphabetic characters to lowercase while preserving spaces and numbers.
- An empty string remains an empty string.

Expected Output

```
-----
Ran 8 tests in <time>

OK
Result: All Task 2 tests pass.
```

5. Task 3 – List Sum Calculator

Function: sum_list(numbers)

Requirements: Empty lists return 0, negative numbers are supported, and non-numeric values are ignored safely.

AI Prompt Used

Generate unittest test cases for sum_list(numbers).
 Requirements:

1. Test a normal list of numbers.
2. Test an empty list.
3. Test negative numbers.
4. Test a list containing strings or other non-numeric values; these values should be ignored.
5. Test a list containing integers and floating-point numbers.
6. Test invalid input when the argument itself is not a list; expect TypeError.

AI-Generated Test Cases

```
import unittest

class TestSumList(unittest.TestCase):
    def test_normal_list(self):
        self.assertEqual(sum_list([1, 2, 3]), 6)

    def test_empty_list(self):
        self.assertEqual(sum_list([]), 0)
```

```

def test_negative_numbers(self):
    self.assertEqual(sum_list([-1, 5, -4]), 0)

def test_ignore_non_numeric(self):
    self.assertEqual(sum_list([2, "a", 3]), 5)

def test_mixed_numeric_types(self):
    self.assertEqual(sum_list([1, 2.5, 3]), 6.5)

def test_invalid_argument(self):
    with self.assertRaises(TypeError):
        sum_list("123")

```

Implementation

```

def sum_list(numbers):
    if not isinstance(numbers, list):
        raise TypeError("Input must be a list")

    total = 0
    for value in numbers:
        if isinstance(value, (int, float)) and not isinstance(value, bool):
            total += value
    return total

```

Code Explanation

- The function verifies that the argument is a list.
- The accumulator total starts at 0, so an empty list naturally returns 0.
- Integer and floating-point values are added.
- Non-numeric values such as strings are ignored rather than causing an error.
- Boolean values are excluded because bool is a subclass of int.

Expected Output

```

-----
Ran 6 tests in <time>

OK
Result: All Task 3 tests pass.

```

6. Task 4 – Student Result Class

Class: StudentResult | Methods: add_marks(mark), calculate_average(), get_result()

Requirements: Marks must be between 0 and 100. Average ≥ 40 means Pass; otherwise Fail.

AI Prompt Used

Generate unittest test cases for a StudentResult Python class with:
add_marks(mark), calculate_average(), and get_result().

Requirements:

1. Marks must be numeric and between 0 and 100 inclusive.
2. Test multiple marks producing Pass.
3. Test multiple marks producing Fail.
4. Test boundary marks 0 and 100.
5. Test invalid negative and greater-than-100 marks; expect ValueError.
6. Test non-numeric marks; expect TypeError.
7. Test calculate_average and get_result when no marks have been added; expect ValueError.

AI-Generated Test Cases

```
import unittest

class TestStudentResult(unittest.TestCase):
    def test_pass_result(self):
        s = StudentResult()
        for mark in [60, 70, 80]:
            s.add_marks(mark)
        self.assertEqual(s.calculate_average(), 70)
        self.assertEqual(s.get_result(), "Pass")

    def test_fail_result(self):
        s = StudentResult()
        for mark in [30, 35, 40]:
            s.add_marks(mark)
        self.assertEqual(s.calculate_average(), 35)
        self.assertEqual(s.get_result(), "Fail")

    def test_boundary_pass(self):
        s = StudentResult()
        s.add_marks(40)
        self.assertEqual(s.get_result(), "Pass")

    def test_boundary_marks(self):
        s = StudentResult()
        s.add_marks(0)
        s.add_marks(100)
        self.assertEqual(s.calculate_average(), 50)

    def test_negative_mark(self):
        s = StudentResult()
        with self.assertRaises(ValueError):
            s.add_marks(-10)

    def test_mark_above_100(self):
        s = StudentResult()
        with self.assertRaises(ValueError):
            s.add_marks(101)

    def test_non_numeric_mark(self):
        s = StudentResult()
        with self.assertRaises(TypeError):
            s.add_marks("80")

    def test_average_without_marks(self):
        s = StudentResult()
        with self.assertRaises(ValueError):
            s.calculate_average()

    def test_result_without_marks(self):
```

```
s = StudentResult()
with self.assertRaises(ValueError):
    s.get_result()
```

Implementation

```
class StudentResult:
    def __init__(self):
        self.marks = []

    def add_marks(self, mark):
        if isinstance(mark, bool) or not isinstance(mark, (int, float)):
            raise TypeError("Mark must be numeric")
        if mark < 0 or mark > 100:
            raise ValueError("Mark must be between 0 and 100")
        self.marks.append(mark)

    def calculate_average(self):
        if not self.marks:
            raise ValueError("No marks available")
        return sum(self.marks) / len(self.marks)

    def get_result(self):
        average = self.calculate_average()
        return "Pass" if average >= 40 else "Fail"
```

Code Explanation

- The constructor creates an empty marks list.
- `add_marks()` validates the type and range before adding a mark.
- Marks outside 0–100 raise `ValueError`.
- `calculate_average()` prevents division by zero by raising `ValueError` when no marks exist.
- `get_result()` calculates the average and returns `Pass` when the average is at least 40; otherwise it returns `Fail`.

Expected Output

```
-----
Ran 9 tests in <time>

OK
Result: All Task 4 tests pass.
```

7. Task 5 – Username Validator

Function: `is_valid_username(username)`

Requirements: Minimum 5 characters, no spaces, and only alphanumeric characters.

AI Prompt Used

Generate unittest test cases for `is_valid_username(username)`.

Rules:

1. Minimum length is 5 characters.
2. Only letters and numbers are allowed.
3. Spaces are not allowed.
4. Test valid usernames.
5. Test usernames shorter than 5 characters.
6. Test usernames containing spaces.
7. Test usernames containing special characters.
8. Test empty string and non-string input; return False safely instead of raising an exception.

AI-Generated Test Cases

```
import unittest

class TestUsernameValidator(unittest.TestCase):
    def test_valid_username(self):
        self.assertTrue(is_valid_username("user01"))

    def test_short_username(self):
        self.assertFalse(is_valid_username("ai"))

    def test_space_in_username(self):
        self.assertFalse(is_valid_username("user name"))

    def test_special_character(self):
        self.assertFalse(is_valid_username("user@123"))

    def test_empty_username(self):
        self.assertFalse(is_valid_username(""))

    def test_exact_minimum_length(self):
        self.assertTrue(is_valid_username("abc12"))

    def test_non_string_input(self):
        self.assertFalse(is_valid_username(12345))
```

Implementation

```
def is_valid_username(username):
    if not isinstance(username, str):
        return False
    return len(username) >= 5 and username.isalnum()
```

Code Explanation

- Non-string input returns False as safe handling.
- `len(username) >= 5` enforces the minimum length.
- `isalnum()` ensures every character is a letter or number.
- Because spaces and symbols are not alphanumeric, they are automatically rejected.

Expected Output

```
-----  
Ran 7 tests in <time>  
  
OK  
Result: All Task 5 tests pass.
```

8. Combined Test Execution

The following single Python file can be used to run all five tasks together.

```
import unittest  
  
# Paste the implementations for Tasks 1-5 here.  
# Paste the five unittest classes here.  
  
if __name__ == "__main__":  
    unittest.main()
```

Expected Combined Output

```
.....  
Ran 37 tests in <time>  
  
OK  
Total expected tests: 7 + 8 + 6 + 9 + 7 = 37. All tests should pass.
```

9. Test Coverage and Analysis

Task	Tests	Important Cases	Validation
1. Even/Odd	7	Even, odd, zero, negative, large integer	TypeError for non-integer
2. Case Converter	8	Empty, mixed case, spaces/numbers	TypeError for invalid input
3. List Sum	6	Empty, negative, mixed numeric/non-numeric	TypeError for non-list
4. Student Result	9	Pass, Fail, boundaries, invalid marks	ValueError/TypeError

5. Username	7	Minimum length, spaces, symbols	False for invalid/non-string
-------------	---	---------------------------------	------------------------------

The AI-generated tests cover normal inputs, boundary conditions, invalid inputs, and error handling. However, AI-generated tests should not be accepted blindly. They must be reviewed against the problem statement. For example, the username task allows a safe False response for invalid input, while the string converter uses TypeError for invalid input. These behaviors are explicitly defined so the implementation and tests agree.

10. Manual Review / Additional Test Cases

- `is_even(1)` should return False.
- `is_even(-5)` should return False.
- `to_uppercase('Hello World!')` should return 'HELLO WORLD!'.
- `sum_list([0, -2, 7])` should return 5.
- `StudentResult` with marks [40] should return Pass because the boundary is inclusive.
- `is_valid_username('abcde')` should return True because it has exactly five alphanumeric characters.
- `is_valid_username('abcd!')` should return False because ! is not alphanumeric.

11. Conclusion

This lab demonstrated Test-Driven Development with AI assistance. Test cases were created before the implementations, and the implementations were then designed to satisfy the expected behavior. The exercises also demonstrated boundary testing, invalid-input handling, exceptions, and safe validation. Using AI can make test generation faster, but human review is necessary to ensure that tests are complete, logically correct, and consistent with the actual requirements.